LECTURE 10

Project Presentations: A Practitioner's Guide

Reference Text

How to frame, build, validate, report and defend a defence-AI project — with the assessment rubric, a model project outline and the STARE method for hard questions

AK DHAMIJA, Researcher & Instructor, AI & Machine Learning

dhamija.ak@gmail.com • 9811453163

Unit 4: Simulation Strategies in Warfare — AI & Machine Learning in Warfare

This reference text expands every slide of the lecture into full explanations, derivations, worked examples and figures. Read it alongside the slides; work the "check your understanding" boxes before looking at the answers.

Contents

1 What the Panel Is Really Assessing **3**

2 Framing the Problem **3**

3 Data: the First Thing the Panel Will Probe **3**

4 Experimental Design **3**

4.1 Baselines . . . 3

4.2 Validation protocol . . . 3

4.3 Ablations . . . 3

5 Reporting Results Honestly **4**

5.1 Metrics that match the decision . . . 4

5.2 Uncertainty . . . 4

5.3 Visualisation . . . 4

6 Statistics for Your Results **4**

6.1 Bootstrap confidence intervals . . . 4

6.2 Comparing RL methods across seeds . . . 4

6.3 Effect sizes . . . 5

7 Failure Modes and Effects Analysis **5**

8 Documentation: Data Cards and Model Cards **5**

9 Reproducibility Checklist **5**

10 Planning the Project **6**

11 Slide-by-Slide Plan for Ten Minutes **6**

12 A Worked Rubric Scoring **6**

13 Expanded Panel Question Bank **7**

14 The Deployment Plan **7**

15 Limitations and Ethics **7**

16 The Assessment Rubric Explained **7**

17 The Ten-Minute Presentation **8**

18 Handling Questions: STARE **8**

19 Common Pitfalls **8**

20 A Model Project Outline **8**

Part C — Research Methods for AI Projects **9**

21 Designing Experiments **9**

21.1 From question to hypothesis 9

21.2 Factorial designs for simulation studies 9

21.3 How many replications? 9

22 Tuning Without Fooling Yourself **9**

23 Statistics for Results, Part II **10**

23.1 How many test cases do you need? 10

23.2 Small samples: the Wilson interval 10

23.3 Many comparisons 10

24 Ablation and Sensitivity Analysis **10**

25 From Notebook to Deployable System **11**

25.1 Machine-learning operations 11

25.2 Latency and the edge 11

26 Threat-Modelling a Machine-Learning System **11**

27 Writing the Technical Report **12**

28 A Fully Worked Example Project **12**

1 What the Panel Is Really Assessing

The project is not a test of algorithm recall but of judgement: did you identify the right problem, choose and justify a method, validate it honestly, and think through how it would be used and how it could fail? Panels include external defence-AI practitioners who ask how your system would perform in real operational conditions.

2 Framing the Problem

A good problem statement is specific, quantified and operational. Weak: "We applied machine learning to air defence." Strong: "IADS operators at a forward air base receive about 400 radar tracks per hour; manual classification takes 45 seconds per track; we need a classifier with at least 95% accuracy and bomber recall above 95% within 3 ms." Name the environment and threat, quantify the cost of not solving the problem, and identify the decision your system supports. Then map it to a paradigm using the three master questions (labels? interaction? multiple agents?).

3 Data: the First Thing the Panel Will Probe

Report N, features, class distribution and how the data were generated (real, exercise, simulation). Then address the Lecture 1 checklist explicitly: collection and survivorship bias; distribution shift between data source and deployment; label quality and noise; class imbalance and the operational base rate; confounders that could produce Simpson-type reversals; leakage (temporal, duplicate, label). A presentation that never mentions data quality is a red flag.

4 Experimental Design

4.1 Baselines

Always compare against (1) a trivial baseline (majority class, random policy, mean predictor), (2) a simple model (logistic regression, linear regression, k-Means), and (3) the relevant course benchmark. An 87% result means nothing without context.

4.2 Validation protocol

Fix the train/validation/test split (stratified, and split by mission or day for sequential data) before modelling; tune only on validation or cross-validation; evaluate the test set once. For RL, train with several random seeds (at least five) and evaluate on many episodes against several opponents.

4.3 Ablations

Remove one component at a time (a feature group, the curriculum, the target network) to show what actually drives performance.

5 Reporting Results Honestly

5.1 Metrics that match the decision

Classification: confusion matrix, per-class precision and recall, and precision at the operational base rate; F_β reflecting error costs. Regression: RMSE, MAE, signed bias and prediction intervals. Clustering: silhouette plus agreement between methods or with experts. Anomaly detection: TPR at a fixed false-alert budget per day. RL: mean and spread across seeds, win rate against several opponents.

5.2 Uncertainty

Report confidence intervals. For an accuracy $\hat{a}$ on n test cases, $\hat{a} \pm 1.96\sqrt{\hat{a}(1-\hat{a})/n}$.

Worked example — is the improvement real?

Your classifier scores 95.0% on 400 test tracks; the course benchmark is 97.5%. The 95% interval for your result is $95.0 \pm 1.96\sqrt{0.95 \times 0.05/400} = 95.0 \pm 2.1\%$, i.e. (92.9, 97.1). The benchmark lies just outside, suggesting your model is genuinely somewhat worse; with only 100 test tracks the interval would be $\pm 4.3\%$ and the comparison inconclusive. For two models on the same test set, use McNemar's test; for RL, compare across seeds.

5.3 Visualisation

One clear chart comparing your result with baselines, a confusion matrix or reliability diagram, and a figure of the system architecture are worth more than many tables.

6 Statistics for Your Results

6.1 Bootstrap confidence intervals

When no formula is convenient (e.g. for F_1, AUC or average reward), resample the test set with replacement many times (say 1,000), recompute the metric on each resample, and take the 2.5th and 97.5th percentiles as a 95% interval. For 93% accuracy on 500 tracks a bootstrap typically gives about (90.8%, 95.2%), matching the normal approximation; for a rare-class recall based on 20 positives it will be much wider — which is exactly what the panel needs to see.

6.2 Comparing RL methods across seeds

Worked example — Welch's t-test

Method A over five seeds: mean success 0.82, s.d. 0.04. Method B: mean 0.78, s.d. 0.05. $SE = \sqrt{0.04^2/5 + 0.05^2/5} = \sqrt{0.00082} = 0.0286$; $t = 0.04/0.0286 = 1.40$, with about 7.6 degrees of freedom (Welch–Satterthwaite), $p \approx 0.20$. A four-point advantage over five seeds is not established; more seeds, or evaluation on more episodes per seed, are needed.

6.3 Effect sizes

Report how big a difference is, not only whether it is significant: absolute and relative differences, and for continuous outcomes Cohen's $d = (\bar{x}_A - \bar{x}_B)/s_{\text{pooled}}$ (0.2 small, 0.5 medium, 0.8 large). In the seed example, $d = 0.04/0.045 \approx 0.9$: potentially large but imprecisely estimated — a reason to run more seeds, not to claim victory.

7 Failure Modes and Effects Analysis

Engineering design reviews list how a system can fail and rate each failure for severity (S), occurrence (O) and detectability (D) on 1–10 scales; the risk priority number RPN $= S \times O \times D$ ranks what to fix first.

Failure mode (anomaly-detection project)	S	O	D	RPN
Novel attack mimics normal traffic and is missed	9	4	7	252
Concept drift raises false alerts beyond analyst capacity	6	7	3	126
Baseline poisoned by slow insider activity	8	3	8	192
Automatic isolation cuts a critical service	9	2	4	72

The table directs effort: mimicry and poisoning (RPN 252, 192) call for red-teaming and peer-group baselines; the high-severity isolation risk is controlled by keeping a human approval step.

8 Documentation: Data Cards and Model Cards

Data card: source and collection process; time period and locations; known gaps and biases (e.g. only detected incidents labelled); class distribution and base rate; labelling procedure and estimated noise; preprocessing; permitted uses and restrictions. **Model card:** intended use and users; out-of-scope uses; training and evaluation data; metrics with confidence intervals, per class and per operating condition; known failure modes; autonomy level and human oversight required; version and date. Panels respond well to projects that include both: they show you know where the system should not be used.

9 Reproducibility Checklist

- Fixed random seeds, and results reported across several seeds.
- Exact data splits saved (by mission or day for sequential data).
- Code, configuration and library versions recorded.
- Preprocessing fitted on training data only (pipelines).
- Hyper-parameters chosen on validation data; the test set used once.
- Every number on a slide traceable to a logged run.

10 Planning the Project

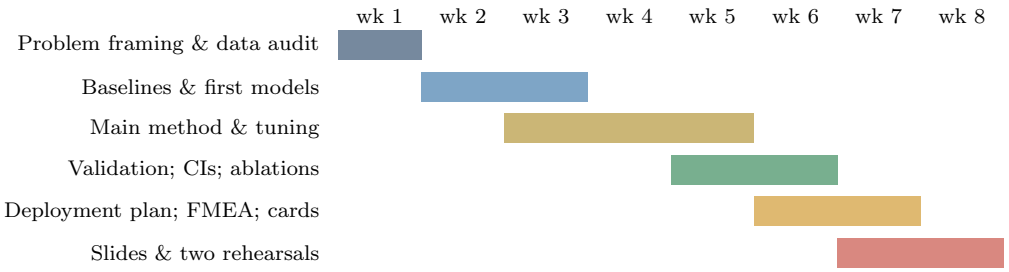


Figure 1: an eight-week project plan. Overlaps are deliberate: start validation before tuning ends.

11 Slide-by-Slide Plan for Ten Minutes

1. Title, team, one-sentence claim.
2. Operational problem with numbers and the cost of inaction.
3. Data: sources, size, class balance, base rate, pitfalls found and fixed.
4. Method: chosen algorithm, two alternatives, why this one (cite course benchmarks).
5. Results 1: headline metric vs baselines, with confidence intervals.
6. Results 2: per-class or per-condition breakdown; confusion matrix or reliability diagram.
7. Ablation or robustness test (shift, noise, adversarial).
8. Deployment concept: architecture, latency, autonomy level, human interface.
9. Risks: FMEA highlights, sim-to-real gap estimate, ethics.
10. Limitations (two specific) and next steps.

12 A Worked Rubric Scoring

Worked example — scoring a sample presentation

A group presents a radar classifier: problem quantified (400 tracks/hour, 3 ms budget) but no cost of inaction stated — *problem 16/20*. Random forest chosen over SVM and logistic regression with course benchmarks cited — *algorithm 22/25*. Accuracy 96.1% with per-class recall but no confidence intervals or baseline — *results 15/25*. Latency and Level-2 operation discussed, sim-to-real not addressed — *operational 14/20*. One generic limitation, no adversarial discussion — *ethics 5/10*. Total **72/100** (weighted by the rubric). The fastest improvements: add confidence intervals and a majority-class baseline (+6–8), quantify the deployment-domain gap (+3–4), and give two specific limitations including adversarial robustness (+3–4).

13 Expanded Panel Question Bank

Data: How were labels obtained, and by whom? What is missing from your data? How does the test set differ from the deployment environment? **Method:** Why not a simpler model? What did the ablation show? **Results:** What is the confidence interval? How many seeds? What does the confusion matrix look like for the rarest class? **Operations:** What does the operator see? What happens when the system is wrong? How will you detect drift? **Adversary:** How would an opponent fool or exploit your system? **Ethics and law:** Who is accountable for a recommendation that leads to harm? Which autonomy level, and why not higher?

14 The Deployment Plan

Separate technically competent projects from operationally mature ones: state the autonomy level (and why), latency budget and hardware, the human–AI interface (what the operator sees, what they approve), ROE and IHL compliance mechanisms, the expected simulation-to-reality gap and how you would close and measure it (domain randomisation, field trials), monitoring for drift after deployment, and the failure modes with their consequences (a short failure-mode and effects table).

15 Limitations and Ethics

Give two specific limitations, not generic ones. Weak: "future work would improve accuracy." Strong: "trained only on L-band signatures; frequency-agile X-band radars would require retraining with domain data." Address adversarial robustness (can an opponent fool or exploit the system?), accountability and IHL where force is involved.

16 The Assessment Rubric Explained

Criterion	Weight	What distinction looks like
Problem definition	20%	Quantified, operationally grounded, cost of inaction stated.
Algorithm justification	25%	Compared with at least two alternatives, citing course metrics; choice explained by data properties.
Results and validation	25%	Per-class metrics, confidence intervals, method, baselines, ablations.
Operational applicability	20%	Autonomy level, latency, ROE, sim-to-real gap quantified, deployment and monitoring plan.
Critical evaluation and ethics	10%	Specific limitations, IHL, adversarial robustness, failure modes.

Peer evaluation contributes 20% of the project grade (clarity, technical depth, operational relevance, question handling), moderated by the instructor.

17 The Ten-Minute Presentation

Problem (60 s) → data and data quality (60 s) → algorithm and justification (90 s) → results and validation (120 s) → deployment plan (90 s) → limitations (30 s), leaving time for transitions. One idea per slide; numbers on every results slide; rehearse to time.

18 Handling Questions: STARE

State what you know (with numbers); **T**ie it to your slides; **A**cknowledge the gap honestly — never bluff; **R**eason about what would happen; **E**nd with what you would do next.

Worked example — a STARE answer

Q: Is your detector robust to adversarial traffic? S: "On 100,000 test packets it achieved 95% TPR at 0.8% FPR." T: "Slide 6 shows the per-attack breakdown." A: "We did not test adversarial perturbations." R: "Autoencoders can be evaded by attacks that imitate normal patterns; R2L is already our weakest class." E: "Next we would red-team with crafted traffic and add an Isolation Forest cross-check."

19 Common Pitfalls

Accuracy-only reporting; no baseline; ignoring data quality; claiming operational readiness from simulation; missing ethics; choosing deep learning by default when a simpler model suffices; misreading statistics (adding percentages, ignoring base rates, "not significant = no effect"); and point results without uncertainty.

20 A Model Project Outline

Title: Tiered anomaly detection for a brigade network. **Problem:** 2 million events/day, analysts can triage 200 alerts; current signature IDS misses novel attacks. **Data:** 30 days of normal traffic plus labelled red-team attacks; checks for leakage by day, base rate 0.01%. **Methods:** Isolation Forest screen, autoencoder confirmation, compared with z-score and one-class SVM baselines. **Results:** TPR at a fixed 150 false alerts/day, with bootstrap confidence intervals; per-attack recall; ablation without tier 1. **Deployment:** edge server, 3 s daily compute, Level-2 containment approvals, drift monitoring and monthly retraining. **Limitations:** mimicry attacks; baseline poisoning; red-team traffic may not represent real adversaries. **Ethics:** privacy of user monitoring, accountability for automated isolation.

Part C — Research Methods for AI Projects

21 Designing Experiments

21.1 From question to hypothesis

A project question ("Can ML improve radar threat classification?") must become a testable hypothesis with a measurable outcome ("A gradient-boosted classifier achieves higher bomber recall at 1% false-positive rate than the current rule set, on held-out data from a later month"). Name the **independent variables** you will change (algorithm, features, sensor set), the **dependent variables** you will measure (recall, latency, false alarms per hour) and the **controls** you hold fixed (data split, preprocessing, compute budget).

21.2 Factorial designs for simulation studies

When several factors may matter, varying one at a time misses interactions and wastes runs. A 2^k **factorial design** runs every combination of k factors at two levels.

Worked example — a 2^2 design

Factor A: sensor package (baseline $-$, upgraded $+$). Factor B: decision algorithm (rules $-$, learned $+$). Mean mission success over replicated simulations: $(-,-) = 70\%$, $(+,-) = 78\%$, $(-,+) = 74\%$, $(+,+) = 90\%$.
Main effect of A $= \frac{78+90}{2} - \frac{70+74}{2} = 12$ points. Main effect of B $= \frac{74+90}{2} - \frac{70+78}{2} = 8$ points. Interaction AB $= \frac{(70+90)-(78+74)}{2} = 4$ points: the learned algorithm gains more when the better sensors are present. A one-factor-at-a-time study starting from $(-,-)$ would have seen gains of 8 and 4 and missed the synergy.

21.3 How many replications?

Simulation outcomes are random, so each configuration is replicated. To estimate a mean to within $\pm E$ with 95% confidence when the run-to-run standard deviation is s, about $n = (1.96\, s/E)^2$ replications are needed. With mission times varying by $s = 8$ minutes and a target precision of ± 2 minutes, $n = (1.96 \times 8/2)^2 \approx 62$ runs per configuration. **Common random numbers** — running each configuration with the same random seeds — make the comparison between configurations far more precise than the individual estimates, because shared luck cancels in the difference.

22 Tuning Without Fooling Yourself

Every hyper-parameter choice made by looking at results is a form of fitting. Three practices keep the final estimate honest:

- **Nested cross-validation:** an inner loop chooses hyper-parameters; an outer loop, which never sees those choices, estimates performance. The test set remains untouched until the end.
- **Equal tuning budgets:** a baseline tuned for an afternoon against a new method tuned for a month is not a fair comparison; give both the same number of trials.
- **Random search over grid search:** when only a few hyper-parameters matter, random sampling explores each important dimension more thoroughly than a grid with the same number of points.

Report the search space, the budget and the selection criterion, so that readers can judge how much optimism the tuning introduced.

23 Statistics for Results, Part II

23.1 How many test cases do you need?

To estimate an accuracy near p to within $\pm E$ at 95% confidence needs about $n = 1.96^2 p(1-p)/E^2$ test cases. For $p = 0.9$ and $E = 0.02$: $n \approx 864$. For a rare class, n counts the cases *of that class*: estimating bomber recall to ± 2 points needs about 864 bombers, not 864 tracks.

23.2 Small samples: the Wilson interval

Worked example — 18 of 20 threats detected

Recall = 0.90. The textbook (Wald) interval $0.90 \pm 1.96\sqrt{0.9 \times 0.1/20}$ gives (0.77, 1.03) — an upper limit above 1, which is impossible. The **Wilson interval**,

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}},$$

gives (0.70, 0.97): honest about how little twenty cases can establish. Use Wilson (or exact Clopper–Pearson) intervals whenever counts are small.

23.3 Many comparisons

Comparing a new model with four alternatives at $\alpha = 0.05$ each inflates the chance of a spurious "win" (Lecture 1). The **Bonferroni** correction tests each at α/m. The **Holm** procedure is uniformly more powerful: sort the p-values, compare the smallest with α/m, the next with $\alpha/(m-1)$, and so on, stopping at the first that fails.

Worked example — Holm versus Bonferroni

Four comparisons with $p = 0.004, 0.014, 0.030, 0.20$. Bonferroni threshold 0.0125: only the first is significant. Holm: $0.004 \le 0.0125$ (reject), $0.014 \le 0.0167$ (reject), $0.030 > 0.025$ (stop). Holm finds two genuine differences while controlling the same family-wise error rate.

24 Ablation and Sensitivity Analysis

An **ablation** removes one component at a time to show what each contributes; a **sensitivity analysis** varies one assumption at a time to show how fragile the conclusion is.

Configuration (illustrative)	Bomber recall	False alarms / hour
Full system (fusion + boosted classifier + calibration)	0.94	1.2
without ESM features	0.86	1.9
without calibration (default threshold)	0.94	3.4
logistic regression instead of boosting	0.89	1.6
trained on 25% of the data	0.88	1.5

A table like this tells the panel which parts of the design matter (here, ESM features for recall and calibration for false alarms) and where effort should go next. Present sensitivity analyses as tornado charts or small tables: "if the base rate doubles, precision rises from 0.16 to 0.28; if ESM is jammed, recall falls to 0.86".

25 From Notebook to Deployable System

25.1 Machine-learning operations

A model that works in a notebook is a prototype. A deployable system also needs: **versioned data and models** (which data trained which model, reproducibly); **automated pipelines** for training, testing and packaging; **monitoring** of input distributions, prediction distributions and, when labels arrive, accuracy; **retraining triggers** and a rollback plan; and documentation (data cards and model cards, as in the main text).

25.2 Latency and the edge

Defence systems often run on constrained hardware at the tactical edge. Every millisecond is budgeted.

Worked example — a latency budget

A requirement of 5 ms from track update to classification: model inference 3.1 ms, preprocessing 0.8 ms, data-link and queueing 1.5 ms — 5.4 ms, over budget. Quantising the model to 8-bit integers roughly halves inference time to about 1.6 ms, giving 3.9 ms within budget, provided accuracy is re-validated after quantisation (it usually drops slightly). Pruning, distillation into a smaller model and hardware accelerators are the other levers.

26 Threat-Modelling a Machine-Learning System

An adversary can attack an ML system at every stage. A project that will face an adversary should include a short threat model:

Attack	What the adversary does	Mitigations
Poisoning	corrupts training data or labels so the model learns a blind spot or backdoor	data provenance, outlier screening, robust training, holding back clean validation data
Evasion	crafts inputs (camouflage, spoofed emissions, adversarial patches) that are misclassified	adversarial training, multi-sensor confirmation, input sanity checks
Model extraction	queries the model to copy it or learn its decision boundary	rate limits, query monitoring, coarse outputs
Inference attacks	recovers sensitive training data from the model's outputs	limit output detail, differential privacy, access control
Supply chain	compromised libraries, pre-trained weights or hardware	signed artefacts, trusted sources, independent testing

27 Writing the Technical Report

A good report follows the same arc as the talk: problem and operational context; data and its limitations; method and why it was chosen over alternatives; results with baselines, uncertainty and ablations; deployment considerations; limitations, risks and ethics; recommendations. Practical rules:

- State the main finding, with its uncertainty, in the first paragraph.
- Every figure has a caption that says what the reader should conclude.
- Every number has units, a sample size and, where meaningful, an interval.
- Distinguish clearly between what was measured, what was inferred and what is speculation.
- Include enough detail (data split, hyper-parameters, seeds, software versions) for someone else to reproduce the results.

28 A Fully Worked Example Project

The numbers below are illustrative; the structure is the point.

Title: Early warning of UAV propulsion faults from telemetry

Problem. A squadron of small UAVs loses about one airframe a quarter to in-flight motor failures. Maintenance is calendar-based. Goal: flag developing faults at least one sortie before failure, with no more than one false alert per 50 sorties.
Data. 18 months of telemetry (motor current, vibration, temperature, battery voltage at 10 Hz) from 2,400 sorties, including 31 sorties preceding a confirmed motor failure. Split by airframe and by time: train on the first 12 months, validate on months 13–15, test on months 16–18.
Baselines. (1) The current calendar schedule; (2) fixed vibration thresholds from the manufacturer.
Method. Per-sortie features (spectral band energies of vibration, current–thrust residuals, temperature rise rate); an Isolation Forest trained on healthy sorties for screening; a CUSUM on each airframe's anomaly score to detect persistent drift (Lecture 5). Thresholds set on the validation months to meet the false-alert budget.
Results (test months). Of 9 pre-failure sorties, 7 were flagged at least one sortie early (recall 0.78, Wilson 95% interval 0.45–0.94); 11 false alerts in 600 healthy sorties (1 per 55). Fixed thresholds caught 3 of 9 with 9 false alerts.
Ablation. Without the CUSUM layer, recall was the same but false alerts doubled; without vibration spectra, recall fell to 4 of 9.
Limitations. Nine failures is a small sample, so the recall interval is wide; all data come from one airframe type and one climate; two missed faults were electrical, not mechanical, and need different sensors.
Deployment. Alerts go to maintenance technicians as advisories with the contributing features displayed; no automatic grounding. Monitor alert rates monthly; retrain quarterly; re-validate after any change of motor supplier.
Ethics and risk. Low direct risk; the main risks are over-trust (skipping inspections because no alert was raised) and alert fatigue. Both are addressed by keeping the calendar inspections during a six-month trial.

Key point

A strong project is honest about what it does not know: a recall of 0.78 with a wide interval, clearly explained, is more persuasive to an experienced panel than an unqualified 0.95.